Day2-DB intuition , SQL

Sunday, 8 February 2026 11:31 PM

What Is Data?

Definition:

Data is raw facts, measurements, or symbols that can be stored, processed, and interpreted.

In computing, it usually means structured (tables, spreadsheets) or unstructured (text, images, logs) information.

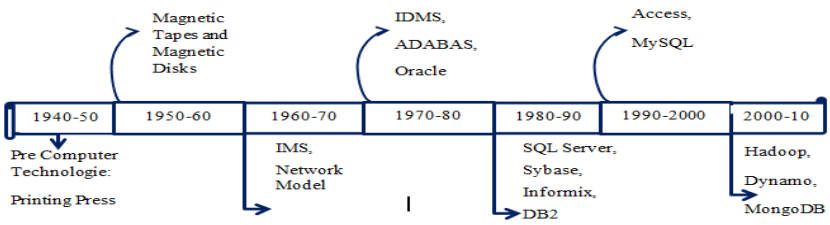
Examples:

- Employee records in a table (name, salary, department_id).
- Click-stream events from a web app (user_id, page, timestamp).
- Sensor readings from a factory (machine_id, temperature, vibration).

Why data matters:

- Data is the source of truth for decisions:
- Business decisions (sales, marketing, HR).
- AI/ML models (predictions, recommendations).
- System monitoring (errors, performance).

History of Databases – File Systems → RDBMS → NoSQL → Big Data → AI



1. File systems (1950s–60s)

   What: Data stored in plain text files, spreadsheets, or CSV-like structures.

   Why it existed: Simple, easy to write and read for small, single-program use cases.

   Problems:

   - No schema or constraints.
   - High redundancy.
   - No relationships between entities.
   - Concurrency issues (multiple programs modifying the same file).

2. Hierarchical & Network Models (1960s–70s)

   Hierarchical:

   Data organized in trees (one parent, many children).

   Example: IBM IMS.

   Network:

   More flexible links between records (graph-like).

   Example: CODASYL.

3. The RDBMS Revolution (1970s–1990s)

   Who: Edgar Codd introduces the relational model.

   What: Data stored in tables, linked by keys and constraints.

   SQL becomes the standard language.

Examples: Oracle, IBM DB2, later MySQL, PostgreSQL, SQL Server.

4. The Rise of the Web (1990s–2000s)

What changed:

Internet, e-commerce, social media, huge user bases → more writes, more reads, more data.

Pain points for RDBMS:

- Vertical scaling (bigger servers) was expensive and limited.
- Complex JOINs on huge tables became slow.
- Need for high write throughput (logs, user events, sessions).

5. NoSQL Era (2000s–2010s)

Goal:

Horizontal scaling across many machines.

Schema flexibility for semi-structured and unstructured data.

High-throughput, low-latency operations.

Why NoSQL emerged:

Relational databases were not built for:

Petabytes of logs.

JSON-like documents (user profiles, sessions, product catalogs).

Real-time, high-volume writes.

6. Big Data & Distributed Systems (2010s)

Hadoop, HDFS, MapReduce:

Distributed file system + batch processing engine.

Designed for massive datasets across clusters.

Apache Spark:

In-memory distributed computation.

Much faster than MapReduce for many workloads.

Why they exist:

Single-machine DBs cannot hold or process petabytes.

Companies need to:

Analyze logs, click-streams, user behavior.

Feed data into ML pipelines.

7. Cloud & Modern Data Platforms (2010s–2020s)

Cloud data warehouses:

Snowflake, BigQuery, Redshift, Databricks.

Managed databases:

Amazon RDS, Aurora; Google Cloud SQL; Azure SQL, Cosmos DB.

8. AI-Native & Vector Databases (2020s–)

Vector databases:

Pinecone, Weaviate, Milvus, Qdrant, etc.

What they store:

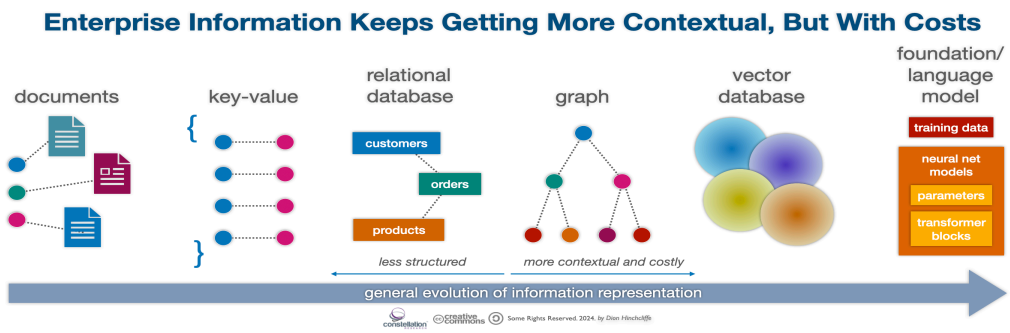
Embeddings (vector representations from ML models).

Use cases:

Semantic search.

Recommendation engines.

RAG (Retrieval-Augmented Generation) systems.



Major Types of Databases

Here's a comprehensive list of database types, why they were created, what data they handle, and when to use them.

Relational (RDBMS)

Examples:

PostgreSQL, MySQL, Oracle, SQL Server, SQLite.

What it is:

Data stored in tables with rows and columns, related via foreign keys and constraints.

When it was created:
1970s (relational model + SQL).

Why it was created:

Solve redundancy and inconsistency in file-based systems.

Provide ACID transactions, constraints, JOINs, and normalization.

Used for banking, HR, ERP, order management, any system where consistency is critical.

Type of data:

Structured, well-defined schemas.

users, products, orders, payments.

When to use:

Financial systems, user accounts, order management.

When you need strong consistency, joins, and transactions.

Backend apps (Django, Spring, Rails) backed by PostgreSQL/MySQL.

Drawbacks:

Scaling joins across many machines is complex.

Schema changes can be slow.

Not ideal for unstructured or semi-structured data (JSON, logs).

3.2. NoSQL: Document Databases

Examples:

MongoDB, CouchDB, CouchBase.

What it is:

Stores data as JSON-like documents (key-value pairs + nested fields).

When created:

Early–mid 2000s.

Why created:

Need schema flexibility for product catalogs, user profiles, CMS.

Need horizontal scaling and high write throughput.

Avoid rigid table schemas.

Type of data:

Semi-structured JSON: user profiles, product catalogs, CMS content, logs.

{"name": "Laptop", "price": 999, "specs": {"ram": "16GB", "cpu": "Ryzen"}, "tags": [...]}.

When to use:

E-commerce product catalogs.

User profiles with many optional fields.

Content management systems.

When you want rapid iteration and schema evolution.

Drawbacks:

Less optimized for complex joins.

Can have eventual consistency trade-offs.

May duplicate data (denormalization).

3.3. NoSQL: Key-Value Stores

Examples:
Redis, DynamoDB, Riak, Memcached.

What it is:
Stores data as key → value mappings.

When created:
Late 1990s–2000s (Redis, DynamoDB, etc.).

Why created:

Need microsecond-level reads/writes.

Perfect for caching, sessions, rate-limiting, job queues.

Simple, scalable model.

Type of data:

Session info: session_id:12345 → {user_id: 101, cart: [...]}.

Counters, flags, caches.

When to use:

Caching layer above RDBMS.

Real-time features: rate-limiting, leaderboards, chat presence.

When you mostly do GET/PUT by key.

Drawbacks:

Not good for complex queries.

Limited querying; you search by key only.

Redis can be memory-expensive for large datasets.

3.4. NoSQL: Column-Family / Wide-Column

Examples:
Cassandra, HBase, ScyllaDB.

What it is:
Data stored by column, not by row. Each row can have many columns that are stored sparsely.

When created:
2000s, inspired by Google Bigtable.

Why created:

Optimized for wide-table, time-series, event-log data.

Need high write throughput and efficient reads for a few columns at a time.

Type of data:

IoT sensor data: device_id, timestamp, temp, humidity, pressure, ...

Click-stream logs.

When to use:

Time-series data, logs, metrics.

When you query by time range and a few columns.

Large-scale analytics that don't need complex joins.

Drawbacks:

Not ideal for ad-hoc JOINs.

Learning curve around partition keys and clustering columns.

Less human-friendly than SQL.

3.5. NoSQL: Graph Databases

Examples:
Neo4j, Amazon Neptune, JanusGraph.

What it is:
Data stored as nodes and relationships (edges).

When created:
2000s, but gained traction in 2010s.

Why created:

Need to model and query highly connected data.

RDBMS JOINs are slow over many relationship hops.

Type of data:

Social networks: User → FRIENDS → User.

Fraud detection: chains of transactions.

Recommendation engines: User → RATED → Product → SIMILAR_TO → Product.

When to use:

Social graphs, recommendation engines.

Fraud detection (linked accounts, transactions).

Anything where you ask "who is connected to whom?".

Drawbacks:

Not great for simple CRUD or transactional workloads.

Can be expensive at very large scale.

Specialized; not a "one-database-solves-all".

3.6. Columnar Databases (Data Warehouses)

Examples:
Google BigQuery, Amazon Redshift, Snowflake, ClickHouse, Vertica.

What it is:
Data stored by column, optimized for analytics and aggregations.

When created:
1990s–2010s; modern cloud versions in 2010s–2020s.

Why created:

RDBMS is row-based, which is fine for transactional systems.

Analytics need fast scans over millions of rows and only a few columns.

Columnar storage:

Better compression.

Fast SUM, COUNT, AVG over large datasets.

Type of data:

Fact tables: user_id, event_type, timestamp, value.

Dimension tables: users, products, geo.

When to use:

Dashboards, BI tools.

Weekly / monthly reports.

ML pipelines that read large feature tables.

Drawbacks:

Slower for point queries (WHERE id = ?).

Not intended for high-frequency OLTP (e.g., INSERT per login).

Cost can be high at huge scale.

3.7. In-Memory Databases

Examples:

Redis, Memcached, SAP HANA, Apache Ignite.

What it is:

Stores data in RAM, not on disk.

When created:

Memcached (early 2000s), Redis (2009), later HANA, Ignite.

Why created:

Disk is slow; RAM is fast.

Need microseconds-level reads/writes for:

Caching.

Session stores.

Real-time leaderboards.

Type of data:

Caches: popular_articles, homepage_banner, user_permissions.

Session data, queues, counters.

When to use:

Caching layer in front of PostgreSQL/MySQL.

Real-time analytics that can fit in RAM.

Situations where latency is more important than capacity.

Drawbacks:

Limited capacity (RAM is expensive).

Volatility (unless persisted to disk).

Not a “source of truth” by itself.

Big Data / Distributed Storage Engines

Examples:

Apache Hadoop (HDFS + MapReduce), Apache Spark.

What it is:

Hadoop: distributed file system (HDFS) + batch processing (MapReduce).

Spark: in-memory distributed processing engine.

When created:

Hadoop ~2006, Spark ~2010

Hadoop ~2006, Spark ~2010.

Why created:

Single-machine databases could not store or process petabytes.

Need to split data across many nodes and process in parallel.

Use cases: logs, click-streams, ETL, batch ML training.

Type of data:

Logs, click-streams, sensor data, raw events.

Parquet, Avro, ORC files stored in HDFS / S3.

When to use:

Batch processing of huge datasets.

Data lakes, data warehouses loading data from HDFS/S3.

Building feature tables for ML.

Drawbacks:

Complexity: clusters, resource management, configuration.

MapReduce can be slow by today's standards.

High infrastructure cost if not managed well.

3.9. Vector Databases

Examples:

Pinecone, Weaviate, Milvus, Qdrant, Chroma.

What it is:

Stores vectors (embeddings) produced by ML models and supports annoy-based or approximate nearest neighbor search.

When created:

2020s, as generative AI and embeddings exploded.

Why created:

RDBMS / document DBs are good at exact matches, not semantic similarity.

Need fast search for "similar items" (products, docs, sentences).

Type of data:

Embeddings from:

Sentence transformers.

Image models (CLIP, etc.).

Product representations.

When to use:

Semantic search (e.g., "find documents about X").

Recommendation engines based on embeddings.

RAG systems: retrieve relevant passages for an LLM.

Drawbacks:

Not for general CRUD.

Requires ML pipeline to generate embeddings.

Often used alongside a relational or document DB.

3.10. Time-Series Databases

Examples:

InfluxDB, TimescaleDB, Prometheus, OpenTSDB.

What it is:

Optimized for time-stamped metrics (metrics, logs, sensor data).

When created:

When created:

2010s–2020s.

Why created:

RDBMS was not optimized for:

Billions of time-stamped events.

Fast aggregations over time windows.

Need for monitoring, alerting, IoT.

Type of data:

Metrics: CPU_usage, memory, latency.

IoT sensor readings.

When to use:

Monitoring dashboards (Prometheus + Grafana).

IoT platforms.

Any system that emits high-frequency time-stamped data.

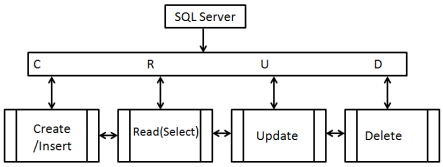
Drawbacks:

Not ideal for complex relational queries.

Limited if you also need rich joins.

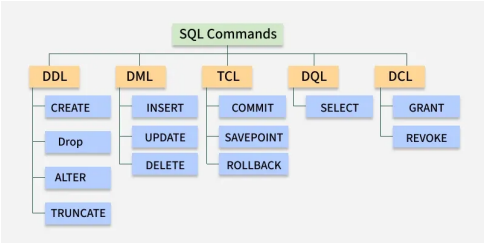
SQL - Structured Query Language:

CRUD in sql:



Concept	SQL Example (employees1)
Create	INSERT INTO employees1 (...) VALUES (...)
Read (all)	SELECT * FROM employees1
Read (subset)	SELECT first_name, last_name FROM employees1
Update	UPDATE employees1 SET salary = ... WHERE employee_id = ?|
Delete	DELETE FROM employees1 WHERE employee_id = ?

Types of language in SQL:



DDL - Data Definition Language -

Command	Description	Syntax
CREATE	Create database or its objects (table, index, function, views, store procedure and triggers)	CREATE TABLE table_name (column1 data_type, column2 data_type, ...);
DROP	Delete objects from the database	DROP TABLE table_name;
ALTER	Alter the structure of the database	ALTER TABLE table_name ADD COLUMN column_name data_type;
TRUNCATE	Remove all records from a table, including all spaces allocated for the records are removed	TRUNCATE TABLE table_name;
COMMENT	Add comments to the data dictionary	COMMENT ON TABLE table_name IS 'comment_text';

RENAME	Rename an object existing in the database	RENAME TABLE old_table_name TO new_table_name;

DQL - Data Query Language

Command	Description	Syntax
SELECT	It is used to retrieve data from the database	SELECT column1, column2, ...FROM table_name WHERE condition;
FROM	Indicates the **table(s)** from which to retrieve data.	SELECT column1 FROM table_name;
WHERE	Filters rows **before** any grouping or aggregation	SELECT column1 FROM table_name WHERE condition;
GROUP BY	Groups rows that have the same values in specified columns.	SELECT column1, AVG_FUNCTION(column2) FROM table_name GROUP BY column1;
HAVING	Filters the results of GROUP BY	SELECT column1, AVG_FUNCTION(column2) FROM table_name GROUP BY column1 HAVING condition;
DISTINCT	Removes **duplicate rows** from the result set	SELECT DISTINCT column1, column2, ... FROM table_name;
ORDER BY	Sorts the result set by one or more columns	SELECT column1 FROM table_name ORDER BY column1 [ASC | DESC];
LIMIT	By default, it sorts in **ascending order** unless specified as DESC	SELECT * FROM table_name LIMIT number;

DML - Data Manipulation Language -

DML commands are used to manipulate the data stored in database tables. With DML, you can insert new records, update existing ones, delete unwanted data or retrieve information.

Command	Description	Syntax
INSERT	Insert data into a table	INSERT INTO table_name (column1, column2, ...) VALUES (value1, value2, ...);
UPDATE	Update existing data within a table	UPDATE table_name SET column1 = value1, column2 = value2 WHERE condition;
DELETE	Delete records from a database table	DELETE FROM table_name WHERE condition;

DCL - Data Control Language -

DCL (Data Control Language) includes commands such as GRANT and REVOKE which mainly deal with the rights, permissions and other controls of the database system. These commands are used to control access to data in the database by granting or revoking permissions.

Command	Description	Syntax
GRANT	Assigns new privileges to a user account, allowing access to specific database objects, actions or functions.	GRANT privilege_type [(column_list)] ON [object_type] object_name TO user [WITH GRANT OPTION];
REVOKE	Removes previously granted privileges from a user account, taking away their access to certain database objects or actions.	REVOKE [GRANT OPTION FOR] privilege_type [(column_list)] ON [object_type] object_name FROM user [CASCADE];

TCL - Transaction Control Language -

Transactions group a set of tasks into a single execution unit. Each transaction begins with a specific task and ends when all the tasks in the group are successfully completed. If any of the tasks fail, transaction fails. Therefore, a transaction has only two results: success or failure.

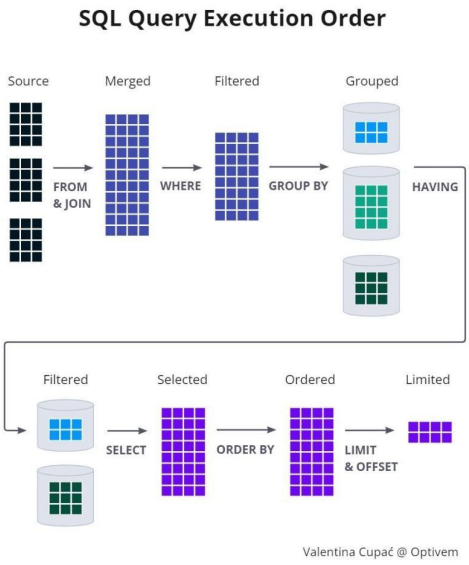
Command	Description	Syntax
BEGIN TRANSACTION	Starts a new transaction	BEGIN TRANSACTION [transaction_name];
COMMIT	Saves all changes made during the transaction	COMMIT;
ROLLBACK	Undoes all changes made during the transaction	ROLLBACK;
SAVEPOINT	Creates a savepoint within the current transaction	SAVEPOINT savepoint_name;

DDL commands have inherent commit onto them

Order of execution in SQL?

1. FROM
2. ON
3. JOIN
4. WHERE
5. GROUP BY
6. HAVING
7. WINDOW (OVER)
8. SELECT
9. DISTINCT

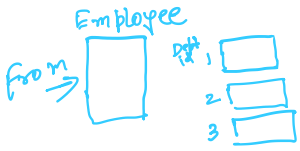
9. DISTINCT
10. UNION / INTERSECT / EXCEPT
11. ORDER BY
12. OFFSET
13. LIMIT / FETCH / TOP



Why should we learn the order of execution in sql?

1. You will be able to resolve nearly most of the errors.
2. You will be able to write better queries
3. You will be able to optimize the queries - run time and interview questions

```
-- Would this query work??
select department_id,min(salary) min_salary,max(salary) as min_salary, avg(salary) avg_salary
from employees
group by department_id
having min_salary > 4000
order by department_id

-- ERROR:  column "min_salary" does not exist
-- LINE 4: having min_salary > 4000
```

min(salary) > 4000